P2591R1
Concatenation of strings and string views

Published Proposal, 2022-05-31

Author:
Giuseppe D'Angelo

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to add overloads of `operator+` between string and string view classes.

Table of Contents

1. Changelog

- R1
  - Minor reading improvements.
- R0
  - First submission.

2. Motivation and Scope

The Standard is currently lacking support for concatenating strings and string views by means of `operator+`:

```
std::string calculate(std::string_view prefix)
{
  return prefix + get_string(); // ERROR
}
```

This constitutes a major asymmetry when considering the rest of `basic_string`'s API related to string concatenation. In such APIs there is already support for the corresponding view classes.

In general, this makes the concatenation APIs between string and string views have a poor usability experience:

```
std::string str;
std::string_view view;

// Appending
str + view;               // ERROR
str + std::string(view); // OK, but inefficient
str + view.data();       // Compiles, but BUG!
```

```
std::string copy = str;
copy += view;            // OK, but tedious to write (requires explicit cop
copy.append(view);       // OK, ditto


// Prepending
view + str;              // ERROR

std::string copy = str;
str.insert(0, view);     // OK, but tedious and inefficient
```

Similarly, the current situation is asymmetric when considering concatenation against raw pointers:

```
std::string str;

str + "hello";    // OK
str + "hello"sv;  // ERROR

"hello"   + str;  // OK
"hello"sv + str;  // ERROR
```

All of this is just bad ergonomics; the lack of `operator+` is extremely surprising for end-users (cf. this StackOverflow question), and harms teachability and usability of `string_view` in lieu of raw pointers.

Now, as shown above, there *are* workarounds available either in terms of named functions (`append`, `insert`, ...) or explicit conversions. However it's hard to steer users away from the convenience syntax (which is ultimately the point of using `operator+` in the first place). The availability of the *other* overloads of `operator+` opens the door to bad code; for instance, it risks neglecting the value of view classes:

```
std::string calculate(std::string_view prefix)
{
  return std::string(prefix) + get_string(); // inefficient
}
```

And it may even open the door to subtle bugs:

```
std::string result1 = str + view; // ERROR. <Sigh>, ok, let me rewrite as..

std::string result2 = str + std::string(view); // OK, but this is inefficie

std::string result3 = str + view.data(); // Compiles; but not semantically
```

The last line behaves differently in case `view` has embedded NULs.

This paper proposes to fix these API flaws by adding suitable `operator+` overloads between string and string view classes. The changes required for such operators are straightforward and should pose no burden on implementations.

§ 2.1. Why are those overloads missing in the first place?

[N3685] ("`string_view`: a non-owning reference to a string, revision 4") offers the reason:

> I also omitted `operator+(basic_string, basic_string_view)` because LLVM returns a lightweight object from this overload and only performs the concatenation lazily. If we define this overload, we'll have a hard time introducing that lightweight concatenation later.

Subsequent revisions of the paper no longer have this paragraph.

There is a couple of considerations that we think are important here.

- `string_view` has been approved for C++17 in Jacksonville (February 2016). At the time of this writing, such a "string builder" facility has not been proposed for standardization (as far as we know). Neglecting a completely reasonable feature to users (concatenation via `operator+`) for so long, in the name of an yet unseen future "major" feature, is a disservice to them.
- We strongly feel that overloading `operator+` is completely outside of the design space for a string builder class. There is absolutely no reason why `str + "hello"sv` should use the builder, but `str + "hello"` should not -- not to mention cases like `strA + strB + strC`. One cannot however change the semantics of the existing `operator+` overloads without breaking API/ABI compatibility. In Qt, [QStringBuilder] uses `operator%` by default; blindly replacing `operator+` with `operator%` when concatenating strings comes with its own share of problems (not only it is API incompatible, but it causes dangling references in a number of scenarios).

In short: we do not see any reason to further withhold the proposed additions.

§ 3. Impact On The Standard

This proposal is a pure library extension.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

§ 4. Design Decisions

The proposed wording builds on top / reuses of the existing one for `CharT *`. In particular, no attempts have been made at e.g. minimizing memory allocations (by allocating only one buffer of suitable size, then concatenating in that buffer). Implementations already employ such mechanisms internally, and we would expect them to do the same also for the new overloads.

The proposed overloads are constrained in the same way as the other string concatenation APIs (e.g. `append`), following the resolution of [LWG2946].

§ 5. Implementation experience

A working prototype of the changes proposed by this paper, done on top of GCC 12.1, is available in this GCC branch on GitHub.

§ 6. Technical Specifications

All the proposed changes are relative to [N4910].

§ 6.1. Feature testing macro

In [version.syn], modify

```
#define __cpp_lib_string_view 201803LYYYYMML // also in <string>, <string_v
```

with the value specified as usual (year and month of adoption of the present proposal).

§ 6.2. Proposed wording

Modify [string.syn] as shown:

```
[...]

// 23.4.3, basic_string

[...]

template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(charT lhs,
              const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(charT lhs,
              basic_string<charT, traits, Allocator>&& rhs);

template<class T, class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const T& lhs,
              const basic_string<charT, traits, Allocator>& rhs);
template<class T, class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const T& lhs,
              basic_string<charT, traits, Allocator>&& rhs);


[...]

template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              charT rhs);
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              charT rhs);


template<class charT, class traits, class Allocator, class T>
  constexpr basic_string<charT, traits, Allocator>
```

```
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              const T& rhs);
template<class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              const T& rhs);
```

Append the following at the end of [string.op.plus]:

```
template<class charT, class traits, class Allocator, class T>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
              const T& rhs);
```

??? *Constraints*:

- `is_convertible_v<const T&, basic_string_view<charT, traits>>` is `true` and
- `is_convertible_v<const T&, const charT*>` is `false`.

??? *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = lhs;
r.append(rhs);
return r;
```

```
template<class charT, class traits, class Allocator, class T>
  constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
              const T& rhs);
```

??? *Constraints*:

- `is_convertible_v<const T&, basic_string_view<charT, traits>>` is `true` and
- `is_convertible_v<const T&, const charT*>` is `false`.

??? *Effects*: Equivalent to:

```
lhs.append(rhs);
return std::move(lhs);
```

```
template<class T, class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const T& lhs,
              const basic_string<charT, traits, Allocator>& rhs);
```

??? *Constraints*:

- `is_convertible_v<const T&, basic_string_view<charT, traits>>` is `true` and
- `is_convertible_v<const T&, const charT*>` is `false`.

??? *Effects*: Equivalent to:

```
basic_string<charT, traits, Allocator> r = rhs;
r.insert(0, lhs);
return r;
```

```
template<class T, class charT, class traits, class Allocator>
  constexpr basic_string<charT, traits, Allocator>
    operator+(const T& lhs,
              basic_string<charT, traits, Allocator>&& rhs);
```

??? *Constraints*:

- `is_convertible_v<const T&, basic_string_view<charT, traits>>` is `true` and
- `is_convertible_v<const T&, const charT*>` is `false`.

??? *Effects*: Equivalent to:

```
rhs.insert(0, lhs);
return std::move(rhs);
```

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[Clazy-qstringbuilder]

auto-unexpected-qstringbuilder. URL: https://github.com/KDE/clazy/blob/master/docs/checks/README-auto-unexpected-qstringbuilder.md

[Libstdcpp-string-concatenation]

operator+ between pointer and string in libstdc++. URL: https://github.com/gcc-mirror/gcc/blob/releases/gcc-12.1.0/libstdc%2B%2B-v3/include/bits/basic_string.tcc#L603

[LWG2946]

Daniel Krügler. *LWG 2758's resolution missed further corrections*. C++20. URL: https://wg21.link/lwg2946

[N3685]

Jeffrey Yasskin. *string_view: a non-owning reference to a string, revision 4*. 3 May 2013. URL: https://wg21.link/n3685

[N4910]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 17 March 2022. URL: https://wg21.link/n4910

[P2591-GCC]

Giuseppe D'Angelo. *P2591 prototype implementation*. URL: https://github.com/dangelog/gcc/tree/P2591_string_view_concatenation

[QStringBuilder]

QStringBuilder documentation. URL: https://doc.qt.io/qt-6/qstring.html#more-efficient-string-construction

[StackOverflow]

Why is there no support for concatenating std::string and std::string_view?. URL: https://stackoverflow.com/questions/44636549/why-is-there-no-support-for-concatenating-stdstring-and-stdstring-view